

Operating System

Prepared By:

Er. Ayush Chaudhary

Chapter 3

Deadlocks

Contents:

Introduction:.....	3
Preemptable & Non-Preemptable Resources:.....	5
Deadlock Characterization or Condition for Deadlocks: ..	7
Resource-Allocation Graph (Modeling deadlock):	8
Method of Handling Deadlocks:	12
Deadlock Ignorance (Ostrich Algorithm):	12
Deadlock Prevention:.....	13
Deadlock Avoidance	14
Deadlock Detection and Recovery from the Deadlock:	21

Introduction:

Deadlock is a situation in which two or more processes are blocked and unable to proceed because they are each waiting for the other to release resources that they need to continue.

This can occur when a process holds a resource and waits for another resource held by a different process, while the other process is also waiting for the first process to release the resource that it holds.



Example 1:

- Two process A and B each want to record a scanned document on a CD.
- A requests permission to use Scanner and is granted.

- B is programmed differently and requests the CD recorder first and is also granted.
- Now, A ask for the CD recorder, but the request is denied until B releases it. Unfortunately, instead of releasing the CD recorder B asks for Scanner. At this point both processes are blocked and will remain so forever. This situation is called Deadlock.

System Model:

Our discussion of deadlock uses a specific system model include the following components:

1. Processes: The entities that are competing for resources in the system.
2. Resources: The objects or assets that are needed by the processes to perform their tasks.
3. Requests: The actions taken by processes to acquire resources.
4. Allocations: The actions taken by the system to grant resources to processes.
5. Wait-for graph: A graphical representation of the dependencies between processes and resources.

Preemptable & Non-Preemptable Resources:

There are two types of resources:

Preemptable: Preemptive resources are those **which can be taken away from a process without causing any ill effects** to the process.

Example: Memory (Process Swapping from main memory)

Non-Preemptable: Non-Preemptive resources are those **which cannot be taken away from the process without causing any ill effects** to the process.

Example: CD-ROM (CD recorder), Printer (Print request by more than one processes).

Under the **normal** mode of operation, a process may **utilize a resource in only the following sequence:**

1. Request: If the request cannot be granted immediately, then the requesting process must wait until it can acquire the resource.
2. Use: The process can operate on the resource.
3. Release: The process releases the resource

S.NO	Deadlock	Starvation
1.	All processes keep waiting for each other to complete and none get executed	High-priority processes keep executing and low-priority processes are blocked
2.	Resources are blocked by the processes	Resources are continuously utilized by high-priority processes
3.	Necessary conditions are Mutual Exclusion, Hold and Wait, No preemption, Circular Wait	Priorities are assigned to the processes
4.	Also known as Circular wait	Also known as lived lock
5.	It can be prevented by avoiding the necessary conditions for deadlock	It can be prevented by Aging

Deadlock Characterization or Condition for Deadlocks:

Deadlock can arise if four conditions hold simultaneously

- **Mutual exclusion:** only one process at a time can use a resource.
- **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes.
- **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task.
- **Circular wait:** there exists a set $\{P_0, P_1, \dots, P_{n-1}\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by P_2 , ..., P_{n-1} is waiting for a resource that is held by P_n , and P_0 is waiting for a resource that is held by P_0 .

Resource-Allocation Graph (Modeling deadlock):

Deadlocks can be described more precisely in terms of a directed graph called a system resource-allocation graph. The set of vertices V is partitioned into two different types of nodes $P = \{P_1, P_2, \dots, P_n\}$ the set consisting of all the active processes in the system; and $R = \{R_1, R_2, \dots, R_1\}$, the set consisting of all resource types in the system.

A directed edge from process P_i to resource type R_j is denoted by $P_i \rightarrow R_j$, it signifies that process P_i requested an instance of resource type R_j and is currently waiting for that resource. A directed edge from resource type R_j to process P_i is denoted by $R_j \rightarrow P_i$ it signifies that an instance of resource type R_j has been allocated to process P_i . A directed edge $P_i \rightarrow R_j$ is called a request edge; a directed edge $R_j \rightarrow P_i$ is called an assignment edge.

When process P_i requests an instance of resource type R_j , a request edge is inserted in the resource-allocation graph. When this request can be fulfilled, the request edge is instantaneously transformed to an assignment edge. When the process no longer needs access to the resource it releases the resource, and as a result the assignment edge is deleted.

Definition of a resource-allocation graph, it can be shown that, if the graph contains no cycles, then no process in the system is deadlocked. If, on the other hand, the graph contains the cycle, then a deadlock must exist.

If each resource type has several instances, then a cycle

implies that a deadlock has occurred. If the cycle involves only a set of resources types, each of which has only a single instance, then a deadlock has occurred. Each process involved in the cycle is deadlocked. In this case, a cycle in the graph is both a necessary and a sufficient condition for the existence of deadlock.

A set of vertices V

and a set of edges E .

V is partitioned

into two types:

- $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the processes in the system.
- $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system.
- request edge – directed edge $P_1 \rightarrow R_j$
- assignment edge – directed edge $R_j \rightarrow P_i$

If each resource type has several instance, then a cycle does not necessarily imply that a deadlock incurred. In this case, a cycle in the graph is a necessary but not a sufficient condition for the existence of deadlock.

The sets P , R and E :

$P = \{P_1, P_2, P_3\}$

$R = \{R_1, R_2, R_3, R_4\}$

$E = \{P_1 \rightarrow R_1, P_2 \rightarrow R_3, R_1 \rightarrow P_2, R_2 \rightarrow P_2, R_2 \rightarrow P_1, R_3 \rightarrow P_3\}$

Resource instances:

- One instance of resource type R_1
- Two instances of resource type R_2
- One instance of resource type R_3
- Three instances of resource type R_4

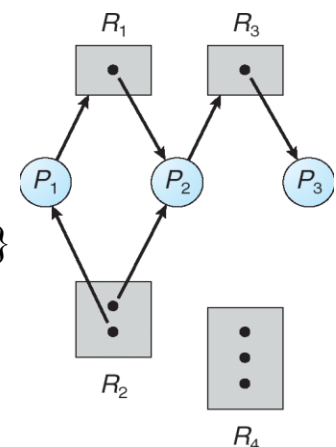


Fig: Resource allocation graph

Process states:

- Process P_1 is holding an instance of resource type R_2 and is waiting for an instance of resource type R_1 .
- Process P_2 is holding an instance of R_1 and an instance of R_2 and is waiting for an instance of R_3 .
- Process P_3 is holding an instance of R_3 .

Suppose that process P_3 requests an instance of resource type R_2 . Since no resource instance is currently available, a request edge $P_3 \rightarrow R_2$ is added to the graph. At this point, two minimal cycles exist in the system:

$P_1 \rightarrow R_1 \rightarrow P_2 \rightarrow$

$R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow$

$P_1 P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow$

$R_2 \rightarrow P_2$

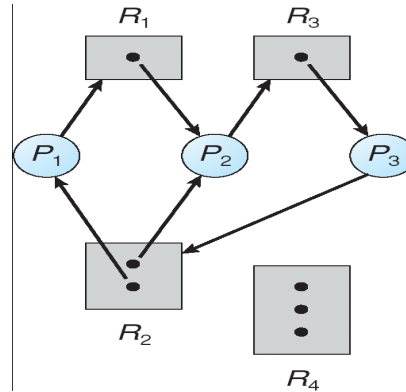


Figure : Resource Allocation Graph with Deadlock

Processes P_1 , P_2 , and P_3 are deadlocked. Process P_2 is waiting for the resource R_3 , which is held by process P_3 . Process P_3 , on the other hand, is waiting for either process P_1 or process P_2 to release resource R_2 . In addition, process P_1 is waiting for process P_2 to release resource R_1 .

We have a cycle in Fig $P_1 \rightarrow R_1 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$

However, there is no deadlock.

Observe that process P_4 may release its instance of resource type R_2 . That resource can then be allocated to P_3 , breaking the cycle.

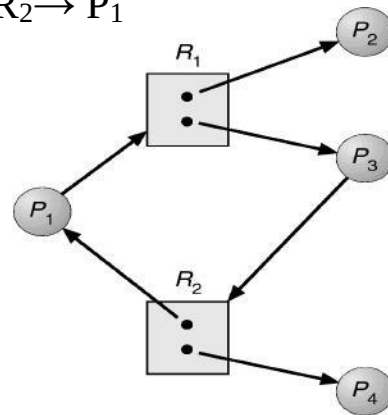


Figure 56: Resource Allocation Graph with cycle but no deadlock

Note:

- If graph contains no cycles $\square$ no deadlock
- If graph contains a cycle $\square$
 - if only one instance per resource type, then deadlock.
 - if several instances per resource type, deadlock possible.
- If a resource-allocation graph does not have a cycle, then the system is not in a deadlocked state.
- If there is a cycle, then the system may or may not be in a deadlocked state

Method of Handling Deadlocks:

There are four approaches to dealing with deadlocks:

1. Deadlock Prevention
2. Deadlock avoidance (Banker's Algorithm)
3. Deadlock detection & recovery
4. Deadlock Ignorance (Ostrich Algorithm)

Deadlock Ignorance (Ostrich Algorithm):

- When storm approaches, an ostrich puts his head in the sand (ground) and pretend (imagine) that there is no Problem at all.
- Ignore the deadlock and pretend that deadlock never occur.
- Reasonable if
 - Deadlocks occur very rarely
 - Difficult to detect
- Cost of prevention is high
- Unix and Windows takes this approach



Deadlock Prevention:

As we know, for a deadlock to occur, each of the four necessary conditions must hold. By ensuring that at least one of these conditions cannot hold, we can prevent the occurrence of a deadlock.

Mutual Exclusion-The mutual-exclusion condition must hold for nonsharable resources. For example, a printer cannot be simultaneously shared by several processes. Sharable resources, on the other hand, do not require mutually exclusive access, and thus cannot be involved in a deadlock

Hold and Wait –must guarantee that whenever a process requests a resource, it does not hold any other resources.

- ◆ Require process to request and be allocated all its resources before it begins execution, or allow process to request resources only when the process has none.

- ◆ Disadvantages: Low resource utilization; starvation possible.

No Preemption –

- ◆ If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released.

- ◆ Preempted resources are added to the list of resources for which the process is waiting.

- ◆ Process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting.

Circular Wait –impose a total ordering of all resource types, and require that each process requests resources in an increasing order of enumeration.

Deadlock Avoidance

- Deadlock avoidance is achieved by being careful at the time of resources allocation.
- The system must be able to decide whether granting a resource is safe or not, and only make the allocation when it is safe.
- Simplest and most useful model: each process declare the maximum number of resources of each type that it may need
- The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition
- Resource-allocation state is defined by the number of available and allocated resources, and the maximum demands of the processes

Safe State

A state is safe if the system can allocate resources to each process (up to its maximum) in some order and still avoid a deadlock. More formally, a system is in a safe state only if there exists a safe sequence. A sequence of processes $\langle P_1, P_2, \dots, P_n \rangle$ is a safe sequence for the current allocation state if, for each P_i the resources that P_j

can still request can be satisfied by the currently available resources plus the resources held by all the P_j , with $j < i$. In this situation, if the resources that process P_i needs are not immediately available, then P_i can wait until all P_j have finished. When they have finished, P_i can obtain all of its needed resources, complete its designated task return its allocated resources, and terminate. When P_i terminates, $P_i + 1$ can obtain its needed resources, and so on.

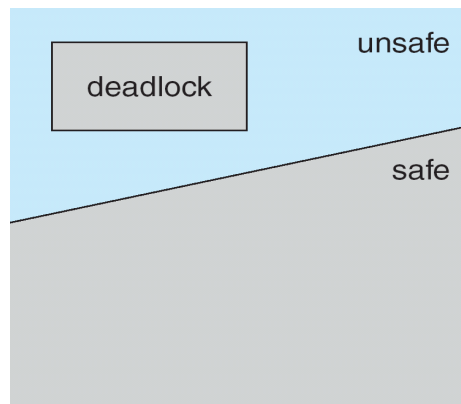


Figure : Safe, Unsafe & Deadlock State

If no such sequence exists, then the system state is said to be unsafe.

Basic Facts:

- If a system is in safe state $\Rightarrow$ no deadlocks
- If a system is in unsafe state $\Rightarrow$ possibility of deadlock
- Avoidance $\Rightarrow$ ensure that a system will never enter an unsafe state

Let's consider that we have altogether 10 resources for 3 processes

Has Max			Has Max			Has Max			Has Max			Has Max		
A	3	9	A	3	9	A	3	9	A	3	9	A	3	9
B	2	4	B	4	4	B	0	—	B	0	—	B	0	—
C	2	7	C	2	7	C	2	7	C	7	7	C	0	—
Free: 3			Free: 1			Free: 5			Free: 0			Free: 7		
(a)			(b)			(c)			(d)			(e)		

Figure: Demonstration that the figure in (a) is safe state

Sequence of run:

1. Process B take 2 resource and leads to figure (b). (Free=1)
2. Process B completes, obtain (c). (Free = 5)
3. Scheduler run process C (Take 5 resource) and leading to (d). (Free= 0)
4. When process completes, get (e). (Free=7)
5. Now process A can get 6 instances.
6. Thus (a) is safe state.

Has Max			Has Max			Has Max			Has Max		
A	3	9	A	4	9	A	4	9	A	4	9
B	2	4	B	2	4	B	4	4	B	—	—
C	2	7	C	2	7	C	2	7	C	2	7
Free: 3			Free: 2			Free: 0			Free: 4		
(a)			(b)			(c)			(d)		

Figure 59: demonstration that the figure in (b) is not safe

Note that: An unsafe state is not a deadlocked state. System can run for a while. From a safe state, the system can guarantee that all processes will finish; from an unsafe state, no such guarantee can be given.

Avoidance algorithms

- Single instance of a resource type
 - o Use a resource-allocation graph
- Multiple instances of a resource type
 - o Use the banker's algorithm

Banker Algorithm:

The banker's algorithm is a resource allocation and deadlock avoidance algorithm that tests for safety by simulating the allocation for predetermined maximum possible amounts of all resources, then makes an "s-state" check to test for possible activities, before deciding whether allocation should be allowed to continue.

Why Banker's algorithm is named so?

Banker's algorithm is named so because it is used in banking system to check whether loan can be sanctioned to a person or not.

Suppose there are n number of account holders in a bank and the total sum of their money is S .

If a person applies for a loan then the bank first subtracts the loan amount from the total money that bank has and if the remaining amount is greater than S then only the loan is sanctioned.

It is done because if all the account holders comes to withdraw their money then the bank can easily do it.

Example:

Consider a system with five processes P0 through P4 and three resources types A, B, C. Resource type A has 10 instances, B has 5 instances and type C has 7 instances. Suppose at time t0 following snapshot of the system has been taken

Process	Allocation	Max	Available
	A B C	A B C	A B C
P0	0 1 0	7 5 3	3 3 2
P1	2 0 0	3 2 2	
P2	3 0 2	9 0 2	
P3	2 1 1	2 2 2	
P4	0 0 2	4 3 3	

1. What will be the content of the need Matrix?
2. Is the system in safe state? If yes, then what is the safe sequence?

1. Need $[i,j] = \text{Max } [i,j] - \text{Allocation}[i,j]$ Content of Need Matrix is

	A	B	c
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1
P4	4	3	1

Applying Safety algorithms

For P_i if $Need_i \leq Available$, then P_i is in Safe sequence,

$Available = Available + Allocation$

For P_0 , $need_0 = 7, 4, 3$

$Available = 3, 3, 2$

$\implies$ Condition is false, So P_0 must wait.

For P_1 , $need_1 = 1, 2, 2$

$Available = 3, 3, 2$

$need_1 < Available$

So P_1 will be kept in safe sequence. & $Available$ will be updated as:

$Available = 3, 3, 2 + 2, 0, 0 = 5, 3, 2$

For P_2 , $need_2 = 6, 0, 0$

$Available = 5, 3, 2$

$\implies$ Condition is again false, so P_2 also have to wait.

For P_3 , $need_3 = 0, 1, 1$

$Available = 5, 3, 2$

$\implies$ Condition is true, P_3 will be in safe sequence. $Available = 5, 3, 2 + 2, 1, 1 = 7, 4, 3$

For P_4 , $need_4 = 4, 3, 1$

$Available = 7, 4, 3$

==> Condition $\text{Need} \leq \text{Available}$ is true, so P4 will be in safe sequence
 $\text{Available} = 7, 4, 3 + 0, 0, 2 = 7, 4, 5$

Now we have two processes P0 and P2 in waiting state. Either P0 or P1 can be chosen. Let us take P2 whose need = 6, 0, 0

$\text{Available} = 7, 4, 5$

Since condition is true, P2 now comes in safe state leaving the Available
 $= 7, 4, 5 + 3, 0, 2 = 10, 4, 7$

Next P0 whose need = 7, 4, 3

$\text{Available} = 10, 4, 7$

Since condition is true P0 also can be kept in safe state.

So system is in safe state & the safe sequence is $\langle P1, P3, P4, P2, P0 \rangle$

Deadlock Detection and Recovery from the Deadlock:

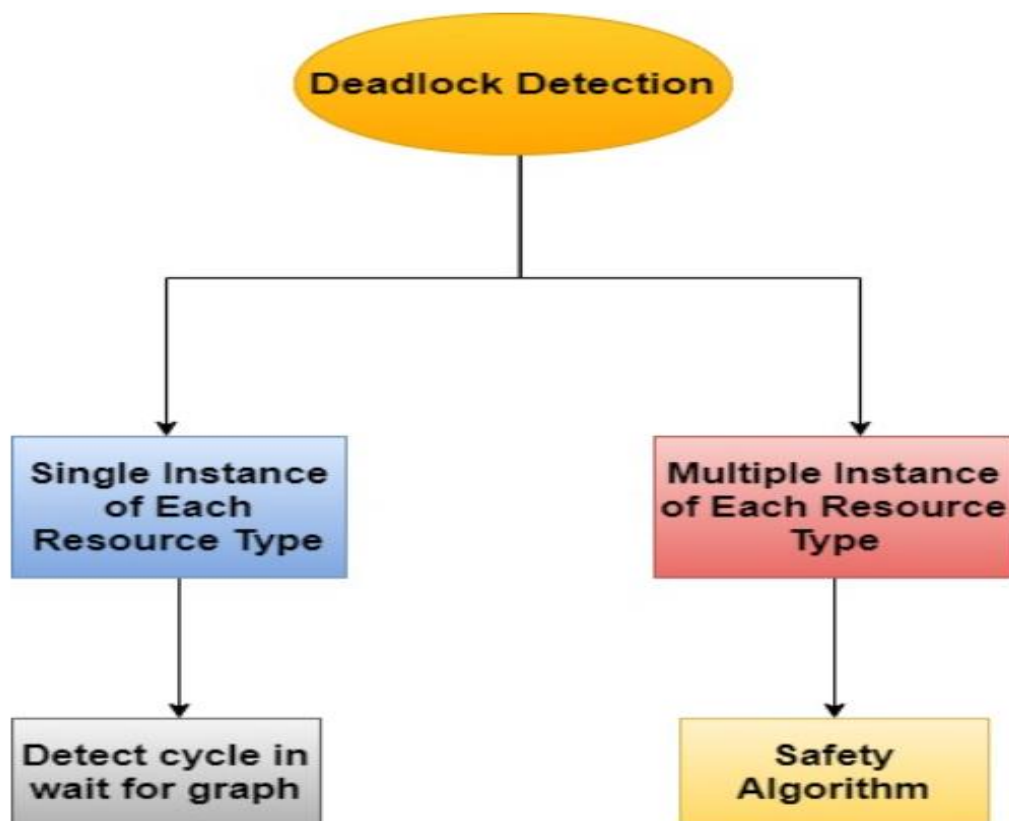
If a system does not employ either a deadlock-prevention or deadlock-avoidance algorithm, then there are chances of occurrence of a deadlock.

In this case, the system may provide two things:

- An algorithm is used to **examines the state of the system** in order to determine **whether a deadlock has occurred**.
- An algorithm that is used to **recover from the deadlock**.

Thus order to get rid of deadlocks the operating system periodically checks the system for any deadlock. After Finding the deadlock the operating system will recover from it using recovery techniques.

Now, the main task of the operating system is to detect the deadlocks and this is done with the help of Resource Allocation Graph.



Single Instance of Each Resource Type

If all the resources have only a single instance, then a deadlock-detection algorithm can be defined that mainly uses the variant of the resource-allocation graph and is known as a wait-for graph.

This wait-for graph is obtained from the resource-allocation graph by removing its resource nodes and collapsing its appropriate edges.

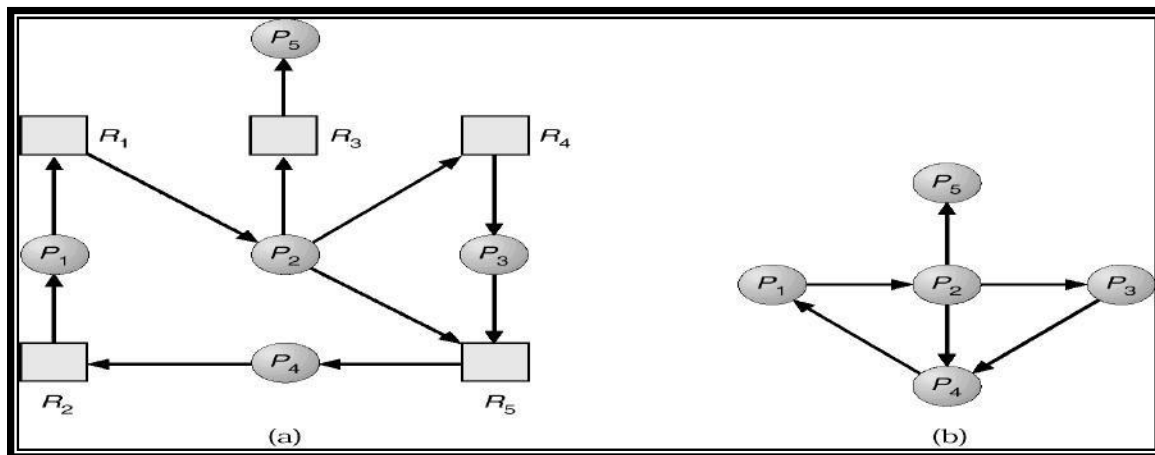


Figure : (a): Resource Allocation Graph

Fig (b): Corresponding Wait for Graph

Multiple Instances of Each Resource Type

The above scheme that is a wait-for graph is not applicable to the resource-allocation system having multiple instances of each resource type. Now we will move towards a deadlock detection algorithm that is applicable for such systems.

This algorithm mainly uses several time-varying data structures that are similar to those used in Banker's Algorithm and these are as follows:

- Available: A vector of length m indicates the number of available resources of each type.

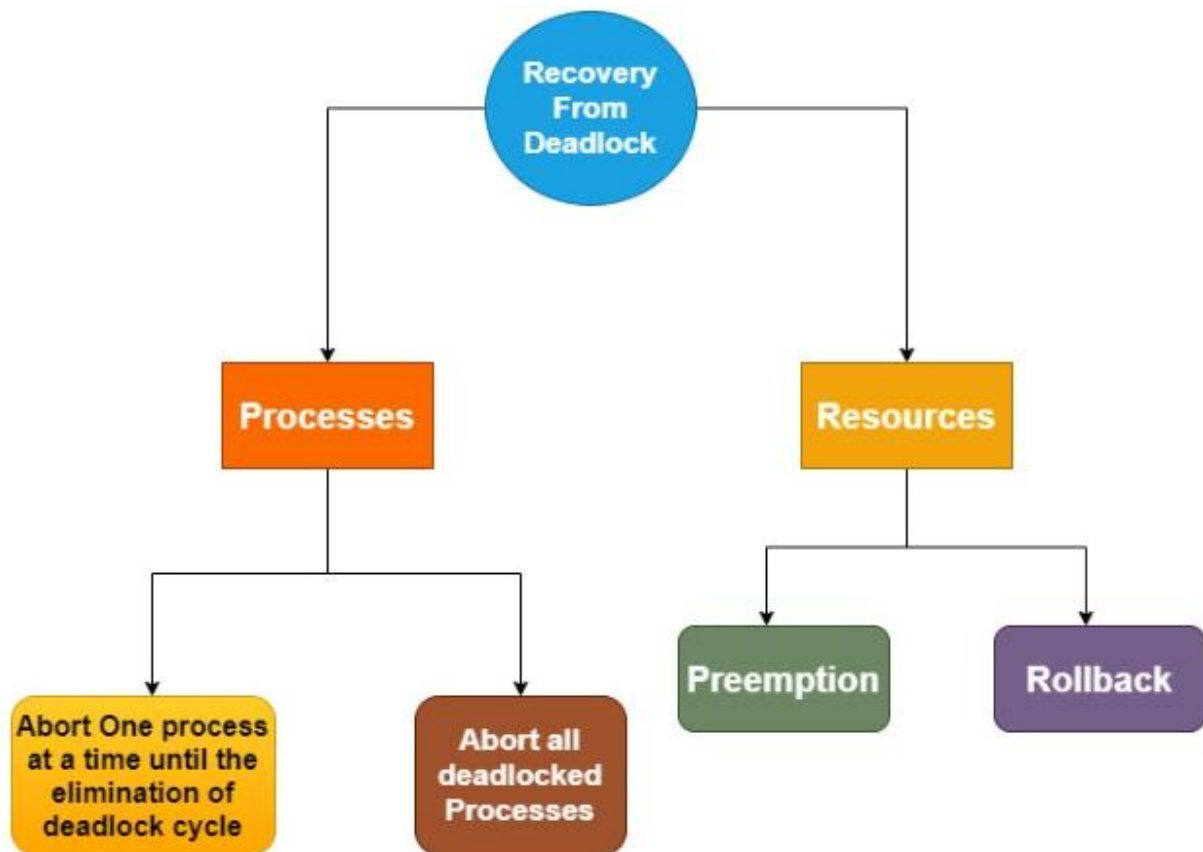
- Allocation: An $n \times m$ matrix defines the number of resources of each type currently allocated to each process.
- Request: An $n \times m$ matrix indicates the current request of each process. If Request $[i, j] = k$, then process P_i is requesting k more instances of resource type R_j .

Recovery From Deadlock

When a detection algorithm determines that a deadlock exists then there are several available alternatives.

There one possibility and that is to inform the operator about the deadlock and let him deal with this problem manually.

Another possibility is to let the system recover from the deadlock automatically. These are two options that are mainly used to break the deadlock.



Process Termination

In order to eliminate deadlock by aborting the process, we will use one of two methods given below. In both methods, the system reclaims all resources that are allocated to the terminated processes.

- **Aborting all deadlocked Processes** Clearly, this method is helpful in breaking the cycle of deadlock, but this is an expensive approach. This approach is not suggestable but can be used if the problem becomes very serious. If all the processes are killed then there may occur insufficiency in the system and all processes will execute again from starting.

- **Abort one process at a time until the elimination of the deadlock cycle** This method can be used but we have to decide which process to kill and this method incurs considerable overhead. The process that has done the least amount of work is killed by the Operating system firstly.

Resource Preemption

In order to eliminate the deadlock by using resource preemption, we will successively preempt some resources from processes and will give these resources to some other processes until the deadlock cycle is broken and there is a possibility that the system will recover from deadlock. But there are chances that the system goes into starvation.

RollBack

In database systems, rolling back is a common technique for breaking deadlocks. When using this technique, the system reverses the transactions of the involved processes to a time before the deadlock. The system must keep a log of all transactions and the system's condition at various points in time to use this method. The transactions can then be rolled back to the initial state and executed again by the system. This approach may result in significant delays in the transactions' execution and data loss.